LAB2 part1 Оцінка часової складності (робочий зошит)

Exported 02.03.2026, 00:43:21

LAB2 Частина 1. Оцінка параметрів функції

Виконавець: **впишіть власне ПІБ**, група **впишіть власну групу**
Викладач: **впишіть хто веде лабораторні заняття**

Нам знадобляться бібліотеки `numpy`, `scipy`, `matplolib`.
Бібліотеки є сторонніми, отже потребують встановлення. Краще у віртуальне оточення. Мали бути встановлені, якщо ні (клітина з імпортуванням виконується з помилкою), то треба встановити. Можна прямо з Jupyter. Тільки не під час пари. Встановлення `matplolib` вимагатиме часу.

Наступну клітину можна розкоментовувати та виконувати тільки, якщо в якості ядра (kernel) вибране віртуальне оточення!

In [1]:

```
#!pip install scipy, matplotlib
```

Якщо встановлювати не в оточення, то хоча б тільки для користувача (з ключем --user).

In [2]:

```
#!pip3.14 install scipy, matplotlib --user    # Linux
```

In [3]:

```
#!py -3.14 -m pip install scipy, matplotlib --user    # Windows
```

The document was converted with **Code-Format**: https://code-format.com/

In [4]:

```
import numpy as np
import scipy, scipy.linalg as la
import matplotlib, matplotlib.pyplot as plt
print(f'numpy {np.__version__}, scipy {scipy.__version__}, matplotlib
{matplotlib.__version__}')
```

```
numpy 2.4.2, scipy 1.17.0, matplotlib 3.10.8
```

Постановка задачі

У результаті кількох експериментів з обчислення швидкодії програми була отримана низка даних вигляду (n, t), де
n — розмір оброблюваних вхідних даних,
t — кількість часу, що була в середньому необхідна для їх обробки.

Дослідник знає, що часова складність програми повинна мати вигляд $T(n) = \Theta(n^a)$. Знайти значення параметра a.

Зауваження. Якщо $T(n) = \Theta(n^a)$, то можна вважати, що $T(n) \approx cn^a$.

Підготовка даних

Спочатку підготуємо "експериментальні" дані. Для цього слід задати значення параметрів `c` та `a` та згенерувати "експериментальні" дані в точках також вручну заданої майже логарифмічної шкали `n`, що містить розміри оброблюваних даних.

В якості шкали використаємо послідовність значень 1, 2, 4, 10, 20, 50, 100, 200, 500, 1000.

Створити змінні `c`, `a` зі значеннями 4.5 та 2.2 відповідно.

In [5]:

```
c = 4.5
a = 2.2
```

Створити багатовимірний масив `n` розмірності 1, який зберігає шкалу значень.

Допоможе функція `np.array` [Документація](#).

▶ Підказка.

The document was converted with **Code-Format**: https://code-format.com/

In [6]:

```
n = np.array([   1,    2,    4,   10,   20,   50,  100,  200,  500,
1000])
n
```

```
array([   1,    2,    4,   10,   20,   50,  100,  200,  500, 1000])
```

Далі треба згенерувати результати експериментів. Для цього напишемо функцію `T_shifted`, що за розміром даних буде повертати кількість часу, схожу на час роботи програми на цих даних. Далі ця функція буде застосовуватись до багатовимірних масивів, тому в ній можна використовувати математичні оператори та математичні функції з бібліотеки `numpy`.

Використаємо, наприклад, таку функцію.

$$T_{shifted}(n) = cn^a + 2.5n^{a-1} + 0.23\ln(n)$$

In [7]:

```
def T_shifted(n):
    return c * n**a + 2.5 * n ** (a-1) + 0.23 * np.log(n)
```

За допомогою функції `T_shifted` згенеруємо результати експериментів та запам'ятаємо їх у змінній `t`.

▶ Підказка.

In [8]:

```
t = T_shifted(n)
t
```

```
array([7.00000000e+00, 2.65794860e+01, 1.08518496e+02, 7.53353861e+02,
       3.36873279e+03, 2.48748869e+04, 1.13663920e+05, 5.20815885e+05,
       3.90327333e+06, 1.79247769e+07])
```

Перехід до логарифмічної шкали та застосування методу найменших квадратів (МНК)

The document was converted with **Code-Format**: https://code-format.com/

Якщо $T(n) \approx cn^a$, то $\log T(n) \approx \log c + a \log n$.

(Основа логарифму значення насправді не має. В обчисленнях можна використовувати натуральний логарифм, що обчислюється функцією `np.log`.)

Введемо позначку $b = \ln c$. Тоді попереднє співвідношення перепишеться у вигляді $\log T(n) \approx b + a \log n$. Оцінку параметрів a та b останнього співвідношення можна здійснити за допомогою МНК (див. у файлі LSM.pdf).

- Перейти до логарифмів розмірів вхідних даних, тобто від n до $x = \ln n$. Отримані значення записати в змінну `x`.
- Перейти до логарифмів результатів експериментів, тобто від t до $y = \ln t$. Отримані значення записати в змінну `y`.

▶ Підказка.

In [9]:

```
x = np.log(n)
y = np.log(t)
```

In [10]:

```
x, y
```

```
(array([0.        , 0.69314718, 1.38629436, 2.30258509, 2.99573227,
        3.91202301, 4.60517019, 5.29831737, 6.2146081 , 6.90775528]),
 array([ 1.94591015,  3.28013972,  4.68692063,  6.62453505,
8.12229193,
        10.12161402, 11.6410013 , 13.16315187, 15.17732607, 16.7016945
]))
```

Створити матрицю x (див. LSM.pdf).

$$X = \begin{pmatrix} 1 & x_0 & 1 & x_1 & \dots & \dots & 1 & x_n \end{pmatrix}$$

▶ Підказки.

The document was converted with **Code-Format**: https://code-format.com/

In [11]:

```
X = np.ones(((x.size),2))
X.T[1] = x
X
```

```
array([[1.        , 0.        ],
       [1.        , 0.69314718],
       [1.        , 1.38629436],
       [1.        , 2.30258509],
       [1.        , 2.99573227],
       [1.        , 3.91202301],
       [1.        , 4.60517019],
       [1.        , 5.29831737],
       [1.        , 6.2146081 ],
       [1.        , 6.90775528]])
```

Створити матрицю Y (див. LSM.pdf)

$$Y = \left(\begin{array}{cccc} y_0 & y_1 & \cdots & y_n \end{array} \right)$$

тобто перетворити вектор y на вектор-стовпчик і запам'ятати в змінній Y.

▶ Підказка.

In [12]:

```
Y = y.reshape(x.size,1)
Y
```

```
array([[ 1.94591015],
       [ 3.28013972],
       [ 4.68692063],
       [ 6.62453505],
       [ 8.12229193],
       [10.12161402],
       [11.6410013 ],
       [13.16315187],
       [15.17732607],
       [16.7016945 ]])
```

The document was converted with **Code-Format**: https://code-format.com/

Оцінку параметрів a та b можна здійснити за допомогою МНК (див. LSM.pdf). Матриця

$$C = \begin{pmatrix} b & a \end{pmatrix}$$

є розв'язком матричного рівняння *де*

$$AC = B, \quad \text{де} \quad A = X^T X, \quad B = X^T Y$$

Створити матриці A та B (див. LSM.pdf).

$$A = X^T X, \quad B = X^T Y$$

▶ Підказки.

In [13]:

```
A = X.T@X
B = X.T@Y
```

In [14]:

```
A, B
```

```
(array([[ 10.        ,  34.31563284],
        [ 34.31563284, 167.6006952 ]]),
 array([[ 91.46458524],
        [420.99652603]]))
```

За допомогою функції `solve` з бібліотеки `scipy.linalg`, що містить засоби лінійної алгебри, розв'язуємо лінійну систему рівнянь, задану в матричному вигляді.

Увага! У більш нових версіях бібліотеки результатом буде не вектор, а вектор-стовпчик.

In [15]:

```
C = la.solve(A, B)
C
```

```
array([[1.77104215],
       [2.14928759]])
```

Слід згадати, що

$$C = \begin{pmatrix} b & a \end{pmatrix} = \begin{pmatrix} \ln c & a \end{pmatrix}$$

The document was converted with **Code-Format**: https://code-format.com/

In [16]:

```
a_est = C[1, 0]
c_est = np.exp(C[0, 0])
```

In [17]:

```
print(f'{c = :}, отримали оцінку {c_est:.3f}, різниця {abs(c-
c_est):.4f}\n\
{a = :}, отримали оцінку {a_est:.3f}, різниця {abs(a-a_est):.4f}')
```

```
c = 4.5, отримали оцінку 5.877, різниця 1.3770
a = 2.2, отримали оцінку 2.149, різниця 0.0507
```

Візуальна оцінка якості наближення

Для цього побудуємо графік знайденої функції та накладемо на нього вхідні дані.

Почнемо з того, що побудуємо функцію `T_estimated`, що обчислює час роботи програми за формулою $T(n) = cn^a$ з використанням знайдених оцінок `a_est`, `c_est` параметрів a та c.

Аналогічно функції `T_shifted`, вона буде застосовна до багатовимірних масивів.

In [18]:

```
def T_estimated(n):
    return c_est * n**a_est
```

На вході у нас були значення часу роботи програми в таких точках

In [19]:

```
n
```

```
array([   1,    2,    4,   10,   20,   50,  100,  200,  500, 1000])
```

Можна бачити, що шкала була нерівномірна (і це було правильно! для тієї задачі, що розв'язувалась). Але для візуалізації будемо будувати графік поточково, тому потрібні значення в проміжних точках.

The document was converted with **Code-Format**: https://code-format.com/

Почнемо з того, що визначимо точки, які використаємо в поточковій побудові графіку. Отриманий масив точок, які рівномірно розташовані по всій шкалі, запам'ятаємо в змінній `scale`.

▶ Підказки

In [20]:
```
scale = np.linspace(np.min(n), np.max(n), 500)
```

З використанням функції `T_estimated` обчислимо розрахункові значення функції часової складності на всій шкалі і запам'ятаємо в змінній `times`.

▶ Підказка.

In [21]:
```
times = T_estimated(scale)
```

Візуалізуємо отримані результати.

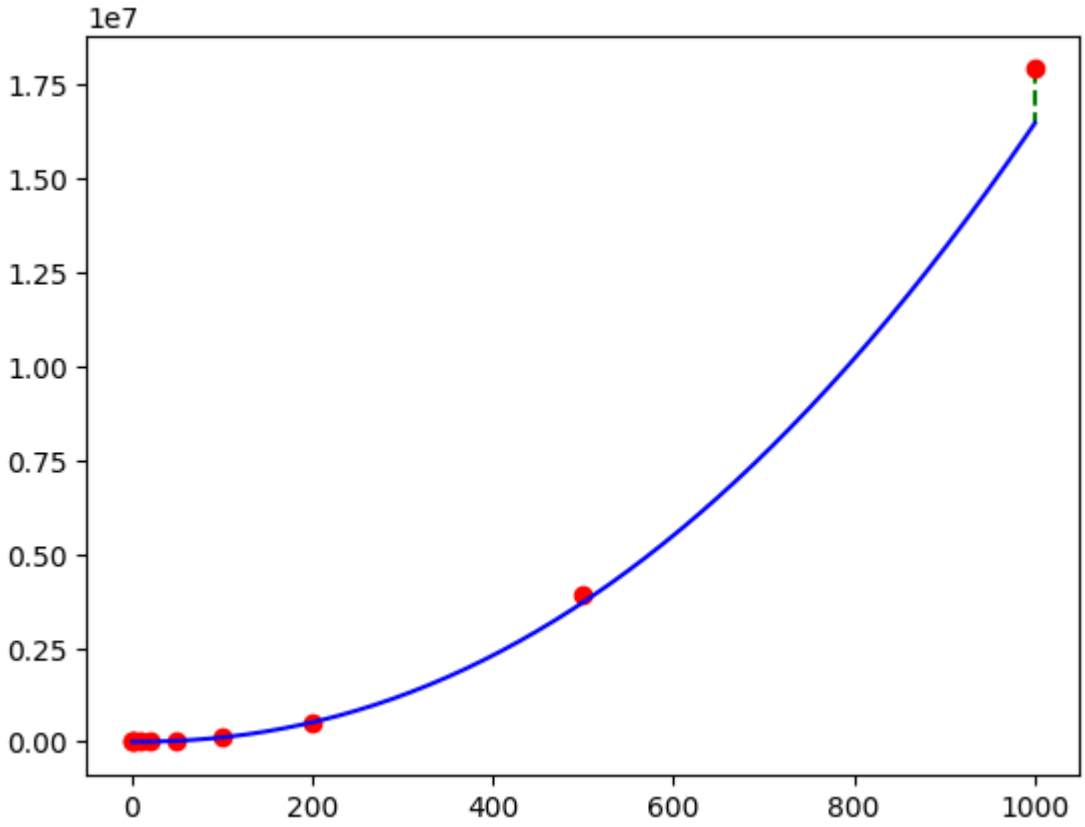
На графіку червоні точки відповідають "експериментальним" даним. Синім зображено графік отриманої залежності часу роботи від розмірів даних.

The document was converted with **Code-Format**: https://code-format.com/

In [22]:

```
for i, px in np.ndenumerate(n):
    py = t[i]
    pz = T_estimated(px)
    plt.plot((px, px),(py, pz), color="green", linestyle='dashed')
plt.plot(n, t, color="red", marker='o', linestyle='None')
plt.plot(scale, times, color='blue')
#plt.show()
#plt.savefig('1.png')
```

```
[<matplotlib.lines.Line2D at 0x265955fd400>]
```



Як зміниться якість наближення, якщо не розглядати експериментальні дані в малих точках?

Частину 1 зроблено!

The document was converted with **Code-Format**: https://code-format.com/